

Titel van de bachelorproef.

Optionele ondertitel.

Elias Van Meerssche.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. T. Clauwaert

Co-promotor: Dhr. B. Van Vreckem

Academiejaar: 2025–2026

Eerste examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Samenvatting

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Inhoudsopgave

Lijst van figuren	vi
Lijst van tabellen	vii
Lijst van codefragmenten	viii
1 Inleiding	1
1.1 Probleemstelling	1
1.2 Onderzoeksvraag	2
1.3 Onderzoeksdoelstelling	2
1.4 Opzet van deze bachelorproef	2
2 Stand van zaken	3
2.0.1 Introductie en Basisconcepten	3
2.0.2 Tracingtechnologieën in Linux	4
3 Methodologie	6
4 Conclusie	8
A Onderzoeksvoorstel	10
A.1 Inleiding	10
A.1.1 Probleemdomein	11
A.1.2 Oplossingsdomein	11
A.2 Literatuurstudie	11
A.2.1 Secure Boot-mechanismen en UEFI-kwetsbaarheden	11
A.2.2 Classificatie, creatie en obfuscatie van Rootkits	12
A.2.3 Kernel-beveiligingsmechanismen en exploit-preventie	12
A.3 Methodologie	13
A.3.1 Iteratie 1: Privilege Escalation	13
A.3.2 Iteratie 2: Technische Verdieping	13
A.3.3 Iteratie 3: Kernel-Module Stealth	14
A.3.4 Iteratie 4: Integratie Automatisering	14
A.4 Verwacht resultaat, conclusie	14
Bibliografie	15

Lijst van figuren

Lijst van tabellen

Lijst van codefragmenten

1

Inleiding

De inleiding moet de lezer net genoeg informatie verschaffen om het onderwerp te begrijpen en in te zien waarom de onderzoeksvraag de moeite waard is om te onderzoeken. In de inleiding ga je literatuurverwijzingen beperken, zodat de tekst vlot leesbaar blijft. Je kan de inleiding verder onderverdelen in secties als dit de tekst verduidelijkt. Zaken die aan bod kunnen komen in de inleiding (**Pollefliet2011**):

- context, achtergrond
- afbakenen van het onderwerp
- verantwoording van het onderwerp, methodologie
- probleemstelling
- onderzoeksdoelstelling
- onderzoeksvraag
- ...

1.1. Probleemstelling

Uit je probleemstelling moet duidelijk zijn dat je onderzoek een meerwaarde heeft voor een concrete doelgroep. De doelgroep moet goed gedefinieerd en afgeijnd zijn. Doelgroepen als “bedrijven,” “KMO’s”, systeembeheerders, enz. zijn nog te vaag. Als je een lijstje kan maken van de personen/organisaties die een meerwaarde zullen vinden in deze bachelorproef (dit is eigenlijk je steekproefkader), dan is dat een indicatie dat de doelgroep goed gedefinieerd is. Dit kan een enkel bedrijf zijn of zelfs één persoon (je co-promotor/opdrachtgever).

1.2. Onderzoeksvraag

Wees zo concreet mogelijk bij het formuleren van je onderzoeksvraag. Een onderzoeksvraag is trouwens iets waar nog niemand op dit moment een antwoord heeft (voor zover je kan nagaan). Het opzoeken van bestaande informatie (bv. “welke tools bestaan er voor deze toepassing?”) is dus geen onderzoeksvraag. Je kan de onderzoeksvraag verder specificeren in deelvragen. Bv. als je onderzoek gaat over performantiemetingen, dan

1.3. Onderzoeksdoelstelling

Wat is het beoogde resultaat van je bachelorproef? Wat zijn de criteria voor succes? Beschrijf die zo concreet mogelijk. Gaat het bv. om een proof-of-concept, een prototype, een verslag met aanbevelingen, een vergelijkende studie, enz.

1.4. Opzet van deze bachelorproef

De rest van deze bachelorproef is als volgt opgebouwd:

In Hoofdstuk 2 wordt een overzicht gegeven van de stand van zaken binnen het onderzoeksdomein, op basis van een literatuurstudie.

In Hoofdstuk 3 wordt de methodologie toegelicht en worden de gebruikte onderzoekstechnieken besproken om een antwoord te kunnen formuleren op de onderzoeksvragen.

In Hoofdstuk 4, tenslotte, wordt de conclusie gegeven en een antwoord geformuleerd op de onderzoeksvragen. Daarbij wordt ook een aanzet gegeven voor toekomstig onderzoek binnen dit domein.

2

Stand van zaken

2.0.1. Introductie en Basisconcepten

Om te begrijpen welke functies in de kernel gebruikt worden om rootkits in te laden moet er begrepen worden waarom deze is in eerste plaats in zitten. Volgens Nagy (2025) zijn er drie manieren om in te laden en twee manieren om de kernel-module te verstoppen. Deze worden hieronder beschreven. De eerste methode is de *Loadable kernel module (LKM)* die normaal gebruikt wordt voor het inladen van drivers. Drivers kunnen op deze manier eenvoudig toegevoegd worden wanneer dit nodig is tijdens het systeem aan staat. Dit zorgt er ook voor dat de kernel niet helemaal opnieuw moet gecompileerd worden om nieuwe modules in te laden. Deze methode kan ook worden gebruikt voor het inladen van rootkits. Na het inladen probeert de module zichzelf te verstoppen om detectie te voorkomen. De tweede methode is *Return-oriented programming (ROP)*, dit kan gebeuren wanneer de aanvaller een memory corruption vulnerability vindt. De derde methode is *Extended Berkeley Packet Filter (eBPF)*. Dit is een eenvoudig alternatief voor LKM's omdat het in een sandbox omgeving runt waardoor het niet een systeemcrash kan veroorzaken, in tegenstelling met LKM's. Omdat een LKM direct in de kernel draait, kan een simpele programmeerfout het geheugen van de kernel of de gebruiker corrumperen. Dit resulteert in een fatale fout dat een kernel panic wordt genoemd, wat het hele besturingssysteem laat vastlopen en een herstart vereist. Het maakt beide gebruikt van Function hooking Mansouri (2026) is een LKM sterk afhankelijk van specifieke kernelversies en moet vaak opnieuw worden gecompileerd voor kleine kernel-updates, omdat interne kernelstructuren per versie verschillen. eBPF is eenvoudiger om mee te werken want het maakt gebruik van *CO-RE (Compile Once, Run Everywhere)*. Hierdoor kan eBPF-code één keer worden gecompileerd en dynamisch op verschillende kernelversies draaien zonder hercompilatie. Een LKM wordt ingeladen als een traditionele module (vaak een fysiek bestand) in het besturingssysteem (Windshock, 2025). eBPF-programma's worden via de `bpfl()`

systeemaanroep direct ingeladen in de eBPF virtuele machine van de kernel en bestaan daardoor niet direct als module-bestand op het bestandssysteem Zaidenberg et al. (2024). Dit maakt eBPF inherent heimelijker (stealthier) dan een LKM. Een LKM heeft onbeperkte toegang en kan direct willekeurige kernelgeheugenlocaties of kernel-datastructuren (zoals de *syscall* table of gebruikersrechten) fysiek overschrijven. eBPF is sterk beperkt: het kan niet zomaar willekeurige functies aanroepen of kernelgeheugen fysiek overschrijven (Mansouri, 2026). eBPF moet altijd gebruikmaken van toegestane helper-functies (zoals *bpf_probe_write_user*) om indirect data of retourwaarden te manipuleren Zaidenberg et al. (2024).

Function hooking tenslotte is een methode om detectie te omzeilen. Als de rootkit ingeladen is, heeft het macht over de originele functies van de kernel. Dit zorgt ervoor dat het deze functies kan overschrijven. Het kan de locatie specifieke bestanden obfusceren waardoor het detectie kan omzeilen. Er is een verschil tussen eBPF's en LKM's. *Direct Kernel Object Manipulation* is een andere techniek om te verbergen dat een kwaadaardige module is ingeladen. Deze methode past de kernelgeheugen aan waardoor het de locatie van bestanden zoals de kernelmodule kan verwijderen (Nagy, 2025).

2.0.2. Tracingtechnologieën in Linux

Volgens Mansouri (2026) heeft LKM veel meer vrijheid met onbeperkte rechten over het systeem. De code in de hook kan willekeurige functies aanroepen en elk deel van het kernel- en gebruikersgeheugen direct lezen of schrijven. Dit is niet zo bij een eBPF. Hierbij is het extreem gelimiteerd wat de code mag doen binnen de hook. In plaats daarvan zijn ze strikt afhankelijk van een specifieke lijst met goedgekeurde eBPF helper-functies om acties en manipulaties uit te voeren. Omdat LKM's directe geheugentoeegang hebben, kapen ze functies vaak op agressievere manieren. Een klassieke LKM-methode is bijvoorbeeld het fysiek en direct overschrijven van pointers in de *syscall* table (de systeemaanroep tabel) van de kernel. In tegenstelling kan eBPF interne kernelstructuren zoals de *syscall* table niet rechtstreeks aanpassen. Om functies te hooken, maakt het uitsluitend gebruik van specifieke, veilig ingebouwde inhaakpunten in de kernel, zoals *Kprobes*, *Uprobes*, en *Tracepoints*. Ook al kan een LKM ook gebruik maken van deze hooks is dit minder nuttig.

Er zijn verschillende hooks, *Uprobes* zijn hooks op functies in de *userland*. Dit zou gebruikt kunnen worden om bepaalde gebruiker programma's functies aan te roepen en aan te passen. *Kprobes* is ook een soort hook maar verschilt van *Uprobes* omdat het functies aanroept binnen de kernel space werken. Er is wel een lijst van functies die hiervoor niet gebruikt mogen worden; de lijst van functies die aangeroepen kan worden staat in */proc/kallsyms*. Het nut van *Kprobes* is om mee te kijken naar wat het systeem aan het doen is. Dit is bedoeld als een debuggingtool. Er bestaan ook nog *Tracepoints* maar deze zijn statisch op voorhand in de kernel

gebakken door de makers.

De kernel heeft een specifieke macro genaamd *notrace*. Als een kernelontwikkelaar dit woord voor een functie zet, vertelt dit de compiler dat deze specifieke functie absoluut niet door ftrace gevolgd (getraced) mag worden. Het is niet mogelijk om functies zoals 'sys_getdents64' *notrace* te maken, omdat grote bedrijven die gebruiken om hun eigen systemen te monitoren.

3

Methodologie

Etiam pede massa, dapibus vitae, rhoncus in, placerat posuere, odio. Vestibulum luctus commodo lacus. Morbi lacus dui, tempor sed, euismod eget, condimentum at, tortor. Phasellus aliquet odio ac lacus tempor faucibus. Praesent sed sem. Praesent iaculis. Cras rhoncus tellus sed justo ullamcorper sagittis. Donec quis orci. Sed ut tortor quis tellus euismod tincidunt. Suspendisse congue nisl eu elit. Aliquam tortor diam, tempus id, tristique eget, sodales vel, nulla. Praesent tellus mi, condimentum sed, viverra at, consectetur quis, lectus. In auctor vehicula orci. Sed pede sapien, euismod in, suscipit in, pharetra placerat, metus. Vivamus commodo dui non odio. Donec et felis.

Etiam suscipit aliquam arcu. Aliquam sit amet est ac purus bibendum congue. Sed in eros. Morbi non orci. Pellentesque mattis lacinia elit. Fusce molestie velit in ligula. Nullam et orci vitae nibh vulputate auctor. Aliquam eget purus. Nulla auctor wisi sed ipsum. Morbi porttitor tellus ac enim. Fusce ornare. Proin ipsum enim, tincidunt in, ornare venenatis, molestie a, augue. Donec vel pede in lacus sagittis porta. Sed hendrerit ipsum quis nisl. Suspendisse quis massa ac nibh pretium cursus. Sed sodales. Nam eu neque quis pede dignissim ornare. Maecenas eu purus ac urna tincidunt congue.

Donec et nisl id sapien blandit mattis. Aenean dictum odio sit amet risus. Morbi purus. Nulla a est sit amet purus venenatis iaculis. Vivamus viverra purus vel magna. Donec in justo sed odio malesuada dapibus. Nunc ultrices aliquam nunc. Vivamus facilisis pellentesque velit. Nulla nunc velit, vulputate dapibus, vulputate id, mattis ac, justo. Nam mattis elit dapibus purus. Quisque enim risus, congue non, elementum ut, mattis quis, sem. Quisque elit.

Maecenas non massa. Vestibulum pharetra nulla at lorem. Duis quis quam id lacus dapibus interdum. Nulla lorem. Donec ut ante quis dolor bibendum condimentum. Etiam egestas tortor vitae lacus. Praesent cursus. Mauris bibendum pede at elit. Morbi et felis a lectus interdum facilisis. Sed suscipit gravida turpis. Nulla at

lectus. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Praesent nonummy luctus nibh. Proin turpis nunc, congue eu, egestas ut, fringilla at, tellus. In hac habitasse platea dictumst.

Vivamus eu tellus sed tellus consequat suscipit. Nam orci orci, malesuada id, gravida nec, ultricies vitae, erat. Donec risus turpis, luctus sit amet, interdum quis, porta sed, ipsum. Suspendisse condimentum, tortor at egestas posuere, neque metus tempor orci, et tincidunt urna nunc a purus. Sed facilisis blandit tellus. Nunc risus sem, suscipit nec, eleifend quis, cursus quis, libero. Curabitur et dolor. Sed vitae sem. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Maecenas ante. Duis ullamcorper enim. Donec tristique enim eu leo. Nullam molestie elit eu dolor. Nullam bibendum, turpis vitae tristique gravida, quam sapien tempor lectus, quis pretium tellus purus ac quam. Nulla facilisi.

4

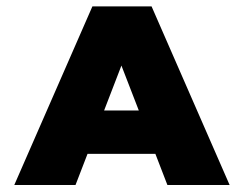
Conclusie

Curabitur nunc magna, posuere eget, venenatis eu, vehicula ac, velit. Aenean ornare, massa a accumsan pulvinar, quam lorem laoreet purus, eu sodales magna risus molestie lorem. Nunc erat velit, hendrerit quis, malesuada ut, aliquam vitae, wisi. Sed posuere. Suspendisse ipsum arcu, scelerisque nec, aliquam eu, molestie tincidunt, justo. Phasellus iaculis. Sed posuere lorem non ipsum. Pellentesque dapibus. Suspendisse quam libero, laoreet a, tincidunt eget, consequat at, est. Nullam ut lectus non enim consequat facilisis. Mauris leo. Quisque pede ligula, auctor vel, pellentesque vel, posuere id, turpis. Cras ipsum sem, cursus et, facilisis ut, tempus euismod, quam. Suspendisse tristique dolor eu orci. Mauris mattis. Aenean semper. Vivamus tortor magna, facilisis id, varius mattis, hendrerit in, justo. Integer purus.

Vivamus adipiscing. Curabitur imperdiet tempus turpis. Vivamus sapien dolor, congue venenatis, euismod eget, porta rhoncus, magna. Proin condimentum pretium enim. Fusce fringilla, libero et venenatis facilisis, eros enim cursus arcu, vitae facilisis odio augue vitae orci. Aliquam varius nibh ut odio. Sed condimentum condimentum nunc. Pellentesque eget massa. Pellentesque quis mauris. Donec ut ligula ac pede pulvinar lobortis. Pellentesque euismod. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent elit. Ut laoreet ornare est. Phasellus gravida vulputate nulla. Donec sit amet arcu ut sem tempor malesuada. Praesent hendrerit augue in urna. Proin enim ante, ornare vel, consequat ut, blandit in, justo. Donec felis elit, dignissim sed, sagittis ut, ullamcorper a, nulla. Aenean pharetra vulputate odio.

Quisque enim. Proin velit neque, tristique eu, eleifend eget, vestibulum nec, lacus. Vivamus odio. Duis odio urna, vehicula in, elementum aliquam, aliquet laoreet, tellus. Sed velit. Sed vel mi ac elit aliquet interdum. Etiam sapien neque, convallis et, aliquet vel, auctor non, arcu. Aliquam suscipit aliquam lectus. Proin tincidunt magna sed wisi. Integer blandit lacus ut lorem. Sed luctus justo sed enim.

Morbi malesuada hendrerit dui. Nunc mauris leo, dapibus sit amet, vestibulum et, commodo id, est. Pellentesque purus. Pellentesque tristique, nunc ac pulvinar adipiscing, justo eros consequat lectus, sit amet posuere lectus neque vel augue. Cras consectetur libero ac eros. Ut eget massa. Fusce sit amet enim eleifend sem dictum auctor. In eget risus luctus wisi convallis pulvinar. Vivamus sapien risus, tempor in, viverra in, aliquet pellentesque, eros. Aliquam euismod libero a sem. Nunc velit augue, scelerisque dignissim, lobortis et, aliquam in, risus. In eu eros. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Curabitur vulputate elit viverra augue. Mauris fringilla, tortor sit amet malesuada mollis, sapien mi dapibus odio, ac imperdiet ligula enim eget nisl. Quisque vitae pede a pede aliquet suscipit. Phasellus tellus pede, viverra vestibulum, gravida id, laoreet in, justo. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Integer commodo luctus lectus. Mauris justo. Duis varius eros. Sed quam. Cras lacus eros, rutrum eget, varius quis, convallis iaculis, velit. Mauris imperdiet, metus at tristique venenatis, purus neque pellentesque mauris, a ultrices elit lacus nec tortor. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent malesuada. Nam lacus lectus, auctor sit amet, malesuada vel, elementum eget, metus. Duis neque pede, facilisis eget, egestas elementum, nonummy id, neque.



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

A.1. Inleiding

De kernel zit in het centrum van het besturingssysteem, en beheert een aantal uiterst belangrijke taken in het systeem, zoals het aansturen van geheugen, processen, hardware en software. De kernel wordt ook als eerste gestart wanneer het besturingssysteem opstart daarna wordt vanuit de kernel het hele systeem aangestuurd. Een rootkit is een soort malware die de aanvaller volledige controle geeft over het systeem wanneer ze uitgevoerd wordt op de kernel die via een kwetsbaarheid in het netwerk of de lokale computer gevonden werd. De kernel regelt alles waardoor de software die van de kernel gebruikmaakt op zijn beurt ook onbetrouwbaar wordt. Omdat een kernel rootkit zo diep in het besturingssysteem zit, is het zeer moeilijk om deze te detecteren.

Het is niet eenvoudig om corruptie van een kernel te demonstreren tijdens de les Cybersecurity Advanced in de hogeschool opleiding Toegepaste Informatica. De lesgever, meneer Clauwaert, wil dat studenten die zich verdiepen in Cybersecurity Advanced aanleren hoe ze een aangetast systeem kunnen herkennen en de schade van een rootkit kunnen beperken. Om dit mogelijk te maken moet er een kernelmodule worden geschreven die als doel heeft een bestand of een proces te verbergen in het besturingssysteem. Dit moet gebeuren zodat de gebruiker op geen enkele manier op de hoogte is van het bestand of proces. De lesgever heeft voor één bepaalde versie van een Linuxdistributie een kernel rootkit nodig die kan worden gebruikt tijdens een laboefening of een troubleshooting test. Het verborgen bestand of proces zal door de studenten gevonden moeten worden tijdens

de oefening. Hoe kan een functionele kernel rootkit worden geïmplementeerd als educatief instrument om studenten Toegepaste Informatica te trainen in het detecteren van kernel corruptie? Het doel van de bachelorproef is om de kernelmodule correct te implementeren, te compileren en succesvol in te laden in het systeem. Om een betere idee te krijgen over hoe de beveiliging werkt op Linux, zullen er enkele deelvragen beantwoord worden.

A.1.1. Probleemdomein

- Welke tekortkomingen heeft de huidige cursus Cybersecurity Advanced op vlak van corruptie detectie in de Linux-kernel?
- Wat is de state of the art van Linux-kernel rootkits?

A.1.2. Oplossingsdomein

Op welke wijze kan de ontwikkelde Loadable Kernel Module (LKM) succesvol in het kernelgeheugen worden geladen, en welke technieken zijn nodig om persistentie te garanderen na een reboot?

- Hoe kunnen beveiligingsmechanismen zoals 'Secure Boot' en 'Kernel Module Signing' in de testomgeving worden geconfigureerd of omzeild om de uitvoering van ongesignde code mogelijk te maken?
- Welke implementatietechniek, zoals *System Call Hooking* of *Direct Kernel Object Manipulation* (DKOM), is het meest effectief om processen en bestanden onzichtbaar te maken voor standaard systeemtools?

A.2. Literatuurstudie

A.2.1. Secure Boot-mechanismen en UEFI-kwetsbaarheden

Het beveiligen van het opstartproces van een computer met Linux-besturingssoftware maakt gebruik van Secure Boot chains. Dit principe werkt op volgende wijze: de *UEFI-firmware* (staat voor Unified Extensible Firmware Interface) bevat standaard de publieke sleutels van Microsoft (Microsoft CA) en vertrouwt daarom binaire bestanden die door Microsoft zijn ondertekend. Dit vertegenwoordigt het 'vertrouwd' onderdeel van de chain (dit noemt Secure Boot). In dit onderdeel staat ook de 'Machine Owner Key Enrollment' of *MOK*, die het mogelijk maakt om eigen componenten toe te voegen aan de Secure Boot chain. Hierbij wordt gebruik gemaakt van eigen sleutels om aanpassingen als legitiem aan te duiden. Hierna wordt *Shim* ingeladen, dit is een pre-bootloader die de schakel vormt tussen *UEFI* en *GRUB2* (de eigenlijke bootloader). Omdat *GRUB2* niet gecertificeerd is door Microsoft, is er een extra schakel nodig in de Secure Boot chain. *Shim* zorgt ervoor dat de Secure Boot chain niet gebroken wordt (Lee et al., 2025). In Lee et al. (2025) wordt

beschreven hoe BootKitty gebruik maakt van verschillende CVE's om de Secure Boot chain te doorbreken en een eigen MOK toe te voegen. Hierdoor wordt *Shim* misleid om de aanpassing als legitiem te valideren. De aanval begint met een 'privilege escalation'. Door specifieke kwetsbaarheden (CVE's) uit te buiten verkrijgen de aanvallers root-rechten. Vervolgens zetten ze een exploit in die gedownload is en geëxecuteerd om de *UEFI-beveiliging* zelf te omzeilen en hun bootkit. Als laatste stap forceren ze een eigen sleutel in de MOK-lijst. Hierdoor wordt het systeem misleid: *Shim* detecteert de toegevoegde MOK, vertrouwt de handtekening, en valideert de kwaadaardige bootkit als legitieme software.

A.2.2. Classificatie, creatie en obfuscatie van Rootkits

Wat meteen opvalt is dat de meeste bronnen over rootkits geschreven zijn over de detectie ervan. Omdat het over schadelijke software gaat is het uiteraard logisch dat er niet veel bronnen bestaan. Er zijn echter wel een aantal dissertaties en artikels te vinden die rootkits classificeren en de verschillende soorten beschrijven (Lacombe et al., 2008). Er zijn in Github ook een aantal lijsten van bestaande rootkits te vinden (skyw4tch3r, 2025) en (Nagy, 2025). Tenslotte worden in dit deel de verschillende methodes bekeken die kunnen worden gebruikt om de rootkits te verstoppen. Volgens Nagy (2025) zijn er drie manieren om in te laden en twee manieren om de kernelmodule te verstoppen. Deze worden hieronder beschreven. De eerste methode is de '*Loadable kernelmodule*' die normaal gebruikt wordt voor het inladen van drivers. Drivers kunnen op deze manier eenvoudig toegevoegd worden wanneer dit nodig is tijdens het systeem aan staat. Deze methode kan ook worden gebruikt voor het inladen van rootkits. Na het inladen probeert de module zichzelf te verstoppen om detectie te voorkomen. De tweede methode is *Return-oriented programming (ROP)*, dit kan gebeuren wanneer de aanvaller een memory corruption vulnerability vindt. De derde methode is Extended Berkeley Packet Filter, waarbij het mogelijk is om bytecode te injecteren in de kernelgeheugen. Dit lijkt op de bovenstaande methodes maar dan zonder kernelmodule. Function hooking tenslotte is een methode om detectie te omzeilen. Als de rootkit ingeladen is, heeft het macht over de originele functies van de kernel. Dit zorgt ervoor dat het deze functies kan overschrijven. Het kan de locatie specifieke bestanden obfusceren waardoor het detectie kan omzeilen. Direct Kernel Object Manipulation is een andere techniek om te verbergen dat een kwaadaardige module is ingeladen. Deze methode past de kernelgeheugen aan waardoor het de locatie van bestanden zoals de kernelmodule kan verwijderen.

A.2.3. Kernel-beveiligingsmechanismen en exploit-preventie

Er bestaan verschillende beveiligingsmechanismen in de Linux-kernel om kernel exploits tegen te gaan. Dit wordt beschreven in Takaronis (2024) en LKMIDAS (2021). Het eerste is *SMEP (Supervisor Mode Execution Prevention)*. Dit voorkomt dat een

aanvaller die een kernel exploit heeft uitgevoerd en macht heeft gekregen over de RIP (Register Instruction Pointer) dit kan gebruiken om de pointer te verwijzen naar een eigen stuk code. *SMEP* zorgt ervoor dat alles in de userland niet uitgevoerd mag worden. De RIP is een CPU-register met het geheugenadres van de volgende instructie. Als iemand dit overneemt kan men de instructie geven om zijn eigen script uit te voeren. Het tweede beveiligingsmechanisme is *SMAP* (*Supervisor Mode Access Prevention*). Dit zorgt ervoor dat geheugen van *userland* niet mag beschreven of gelezen mag worden. Deze beveiliging bestaat omdat aanvallers gebruik maken van ROP-gadgets. Deze bevatten stack-pointers die verwezen kunnen worden naar wat de aanvaller beheerst. De aanvaller plaatst dit dan in de *userland*, *SMAP* voorkomt dan dat de kernel kan lezen of uitvoeren. *KASLR* (*Kernel address space layout randomization*) is een derde beveiligingsmechanisme dat ervoor zorgt dat de geheugenlocaties van de kernel random bepaald worden bij het booten van het systeem. Tenslotte zorgt *KPTI* (*Kernel Page-Table Isolation*) ervoor dat de page tables verdeeld zijn over user page tables en kernel page tables. De verschillende beveiligingsmechanismen zullen omzeild moeten worden door de exploits.

A.3. Methodologie

A.3.1. Iteratie 1: Privilege Escalation

Er wordt gestart met het selecteren van bronnen die concrete voorbeelden of scripts bevatten om root-rechten te verkrijgen. Deze exploits worden direct op het testsysteem uitgevoerd. Er zal getest worden op een virtuele machine. Pas wanneer dit succesvol is, wordt er in detail gekeken naar de technische werking van de kwetsbaarheid. Hierbij wordt specifiek onderzocht of de werking op een educatieve manier kan worden uitgelegd. Mochten de geselecteerde bronnen onvoldoende zijn om daadwerkelijk toegang te krijgen, dan wordt er direct gezocht naar alternatieve kwetsbaarheden. Het doel van deze iteratie is het realiseren van een werkende exploit.

A.3.2. Iteratie 2: Technische Verdieping

Na het succesvol uitvoeren van de aanval op het systeem zal er uitgewerkt worden hoe kwetsbaarheden echt werken. Dit gebeurt op basis van bestaande bronnen. Indien nodig wordt er gezocht naar aanvullende informatie. De bevindingen worden vervolgens verwerkt in een uitgebreid verslag. Het doel is om aan te tonen dat de werking begrepen wordt en om dit vervolgens eenvoudig aan de studenten uit te leggen.

A.3.3. Iteratie 3: Kernel-Module Stealth

Na het succesvol uitleggen van de exploit zal dit gebruikt worden om een Linux-kernelmodule in te laden in het systeem. Deze module heeft als doel om zichzelf te verbergen maar ook het verbergen van processen en bestanden. Het doel van deze iteratie is een geschikte kernelmodule vinden en inladen in het systeem. Hierbij wordt specifiek geanalyseerd hoe de module de kernel beïnvloedt (bijvoorbeeld via syscall hooking) en welke specifieke beveiligingsmechanismen zijn omzeild om dit mogelijk te maken.

A.3.4. Iteratie 4: Integratie Automatisering

In de laatste iteratie worden de exploit en de kernelmodule samengevoegd tot één werkend Proof of Concept. De volledige aanval wordt gedocumenteerd, met nadruk op de impact op het opstartproces en de kernel-beveiliging. Als er nog tijd over is, wordt onderzocht hoe de aanval geautomatiseerd kan worden, bijvoorbeeld via een script of een USB-stick, zodat handmatige configuratie geminimaliseerd wordt. Tot slot wordt de conclusie van het onderzoek geformuleerd.

A.4. Verwacht resultaat, conclusie

Het eindresultaat is een script dat verschillende kwetsbaarheden gebruikt om een Linux-kernelmodule in te laden in het systeem zonder dat deze door het systeem als gevaarlijk wordt gedetecteerd. De kernelmodule zorgt ervoor dat processen en bestanden verborgen kunnen worden voor de gebruiker en kan in een leeromgeving worden gebruikt om studenten praktische ervaring met dit soort malware te geven. De lesgever kan via een virtuele machine een omgeving opzetten waarmee studenten kunnen proberen om de rootkit te detecteren en te leren hoe deze eruit ziet in de praktijk.

Bibliografie

- Lacombe, E., Raynal, F., & Nicomette, V. (2008). Rootkit modeling and experiments under Linux. *Journal in Computer Virology*, 4(2), 137–157. <https://link.springer.com/article/10.1007/s11416-007-0069-6>
- Lee, J., Kwon, J., Seo, H., Lee, M., Seo, H., Jung, J., & Koo, H. (2025). {BOOTKITTY}: A Stealthy {Bootkit-Rootkit} Against Modern Operating Systems. *19th USENIX WOOT Conference on Offensive Technologies (WOOT 25)*, 303–320. <https://www.usenix.org/system/files/woot25-lee.pdf>
- LKMIDAS. (2021, januari). *Linux Kernel Pwn: Part 1 - An introduction to Linux Kernel Exploitation*. Geraadpleegd op 12 december 2025, van <https://lkmidas.github.io/posts/20210123-linux-kernel-pwn-part-1/>
- Mansouri, Z. (2026). *Analyzing eBPF Rootkits through the MITRE ATTCK Framework*. <https://cylab.be/publications/90/2026-analyzing-ebpf-rootkits-through-the-mitre-attck-framework>
- Nagy, R. (2025). Detecting hidden kernel modules in memory snapshots. *Forensic Science International: Digital Investigation*, 53, 301928. <https://doi.org/https://doi.org/10.1016/j.fsidi.2025.301928>
- skyw4tch3r. (2025). *RootKits-List-Download: A collection of rootkit links*. Geraadpleegd op 12 december 2025, van <https://github.com/skyw4tch3r/RootKits-List-Download>
- Takaronis, M. (2024). *Linux kernel exploitation, a wonderful mess* [masterscriptie, Πανεπιστήμιο Πειραιώς]. https://doi.org/http://dx.doi.org/10.26267/unipi_dione/3917
- Windshock. (2025, 29 april). *Detection Frameworks and Latest Methodologies for eBPF-Based Backdoors*. Windshock's Blog. Geraadpleegd op 22 april 2026, van <https://windshock.github.io/en/post/2025-04-29-ebpf-backdoor-detection-framework/>
- Zaidenberg, N., Kiperberg, M., Menachi, E., & Eitani, A. Detecting eBPF Rootkits Using Virtualization and Memory Forensics. In: *In Proceedings of the 10th International Conference on Information Systems Security and Privacy - Volume 1: ICISSP*. INSTICC. SciTePress, 2024, 254–261. ISBN: 978-989-758-683-5. <https://doi.org/10.5220/0012470800003648>